Baton

An evidence-graded handover for small software teams. Every statement is linked to its source, every gap is flagged, and nothing reaches the successor until a human approves it.

01 Understanding of the problem
02 The specific problem we solve
03 Users and departure scenarios
04 Evidence that the problem is real
05 Proposed solution
06 Complete product workflow
07 Main features
08 Technology stack and rationale
09 Architecture and data model
10 Major technical challenges
11 Privacy, security and trust
12 Development plan (3 rounds)
13 Expected impact
14 Future scope and limitations
15 Team, role and contribution
16 Disclosures and references

PARTICIPANT
Archit Dandavate

TEAM
LoneWolf · solo (1 member)

ROLE
Full-stack & AI engineer

PROBLEM

01 Understanding of the problem statement

When someone leaves a project, the files usually stay but the **reasoning** goes with them. A successor inherits a repository, a chat history and some notes, and still cannot answer basic questions: why was this built this way, what was left unfinished, who depends on it, and who do I ask?

What actually gets lost

- **Decision rationale.** The *why* lives in chat threads and PR comments, not in docs.
- **Unfinished work.** Half-done branches, open PRs and promises made in chat.
- **Dependencies and contacts.** Which external service, account or person the project relies on.
- **Recurring problems.** The bug that returns every release and the workaround everyone forgot.
- **Ownership.** Who really maintains the deploy script, as opposed to who is listed.

Why existing documentation fails

- **It is written once and goes stale.** Decision records are widely recommended but rarely kept up.
- **It is written at the worst time.** Handover notes are rushed during a notice period or exam week.
- **It records what, not why.** The reasoning is in conversations nobody consolidates.
- **It cannot be trusted blindly.** A summary with no sources gives the successor no way to tell fact from guess.

Our reading of the brief. Summarisation alone is not enough to solve this. The brief's trust requirements say the system must never present assumptions as verified truth, never expose confidential content just because it was in a source, and always let a responsible human correct, reject and approve. We treat these requirements as the core of the product, not as extras.

02 The specific problem we solve

Baton turns a departing member's project activity into a **verified, role-appropriate continuation plan** for **small software project teams** (2–20 people working from GitHub, a team chat and shared notes).

In scope

Reconstructing decisions, ownership, open work, risks, dependencies, contacts and next actions from a GitHub repository, uploaded notes and documents, and a team-chat export. Every claim is evidence-linked and reviewed before sharing.

Out of scope (deliberately)

HR offboarding, access deprovisioning, payroll and exit surveys. Live enterprise connectors at scale, and audio or video meeting processing (text transcripts and notes only).

Why this slice

Turnover in these teams is frequent and predictable (graduation, internships, notice periods). Their sources are standard and text-based, and realistic demo data can be built legally from public repositories plus clearly labelled synthetic chat.

USERS

03 Who experiences the problem

Baton has four roles. The **successor** is the one who benefits, but the **reviewer** is the one who makes the output trustworthy.

Role	Example	What they need from Baton
Leaver outgoing owner	Final-year lead of a college club's app; engineer serving notice at a 10-person startup; OSS maintainer stepping back.	A handover that takes minutes, not days, and asks them only about what the system could not verify.
Successor incoming member	Junior club member taking over the project; new hire; co-maintainer.	Knowing what to do first, who owns what, what is risky, and where each answer came from.
Reviewer approver	Project lead, club coordinator or faculty mentor, engineering manager.	Checking evidence quickly, correcting or rejecting claims, controlling what the successor can see, then approving.
Admin	Whoever set up the workspace.	Connecting authorised sources and setting access per member.

Departure scenarios Baton must handle

Planned exit
Graduation, internship end, resignation with notice. The leaver is available, so Baton drafts first and sends the leaver a short list of targeted questions.

Unplanned exit
Sudden departure, illness, dropping out mid-semester. The leaver is *not* available, so the same unanswered questions go to the reviewer or the closest collaborator, and gaps stay visibly marked.

Role rotation
A member moves to another sub-team but stays reachable. Baton produces a partial handover for one area (e.g. "payments module") rather than the whole project.

Environments: student tech clubs and college project teams, hackathon teams that continue after the event, open-source maintainer groups, and early-stage startups. All of these typically run on GitHub plus Discord or Slack plus shared notes.

04 Evidence that the problem is real

Desk research from published surveys and peer-reviewed studies. Numbers in brackets refer to the references on the last page.

42%
of institutional knowledge is unique to the individual who holds it, according to a survey of over 1,000 US workers [1]

5.3 h/wk
lost per knowledge worker waiting for colleagues' knowledge or recreating it [1]

61%
of developers spend more than 30 minutes a day searching for answers; 30% are hit by knowledge silos 10+ times a week [2]

27%
of open-source projects that lost all core developers managed to attract a new one (study of 36,000+ projects) [3]

- **Loss of the core team is common.** About 89% of the studied open-source projects lost their entire core team at least once, 70% of them within the first three years. Most projects depend on a single core developer [3].
- **Turnover is constant in India's software industry.** Large IT services firms reported voluntary attrition of roughly 12–14% in FY26 [4]. Student clubs face near-total leadership turnover every year, as graduation removes members and officers [10].
- **The reasoning exists, but in chat.** In one study, rationale appeared in about 25% of developer chat messages, and ML could detect it with 0.76 precision and 0.79 recall [5]. The knowledge is recoverable; nobody consolidates it.
- **Plain AI summaries are not trustworthy.** An audit of generative search engines found only 51.5% of generated statements were fully supported by their citations [6]. A handover tool must verify citations rather than assume them.

Primary user research (interviews with club leads and recently onboarded members) is planned for the start of Round 2 and will be reported in the Round 2 document. This proposal relies on desk research only.

SOLUTION

05 Proposed solution: a claim ledger, not a summary

Baton does not generate a handover *document*. It builds a **ledger of atomic claims**, each one a single decision, owner, risk, dependency, contact, recurring issue, open question or next action. Each claim carries its evidence and a status. The handover is then a reviewed, filtered view of that ledger.

Five evidence statuses, assigned by rules rather than by the AI

Verified
Supporting evidence passed the check *and* a human confirmed it.

Supported
Evidence passed the automated check; not yet confirmed by a person.

Inferred
AI reasoning with no passing evidence. Never shared as fact unless a human promotes it with a note.

Conflicting
Sources disagree and neither clearly supersedes the other. Both are shown side by side.

Missing
Something a successor needs, but no source covers it. Becomes a question.

The language model **proposes** claims and the evidence it thinks supports them. Deterministic code and a separate verification model **decide** the status. The model never grades its own work.

What makes Baton different

1 · Detects absence, not just content

From the repository's structure, Baton builds an **expected-knowledge checklist**: every open PR or issue the leaver owns, every external service in `.env.example` or config, every CI workflow, every dependency the leaver mostly maintained. Anything on that list without evidence becomes a MISSING item. A summariser can only describe what it sees; Baton also reports what it should have seen but didn't.

2 · Gap interview, not an exit interview

Instead of a long, generic interview, the leaver gets a **short list of targeted questions** generated only from Missing and Conflicting items. Example: "PR #88 and the 12 March chat disagree on the upload limit. Which one is current?" Answers are stored as a new, cited source labelled *leaver testimony*.

3 · Successor-specific visibility

The leaver could see more than the successor can. When a claim is shared, Baton checks it against the successor's access. A claim that rests on a source the successor cannot open is **withheld**, and the successor sees only a stub ("Restricted item: ask the reviewer") if the reviewer allows it. Before approving, the reviewer sees a preview of exactly what will be visible.

4 · A handover that closes

The successor **acknowledges** each next action and open question. Unresolved items stay open and visible to the reviewer. Every edit, rejection and approval is versioned, so the handover is auditable rather than a static file.

Example output (synthetic project: a college club's events app)

Type	Claim	Evidence	Status
Decision	Push notifications moved from OneSignal to Firebase Cloud Messaging in March because of the free-tier device cap.	PR #88 description · #dev chat, 12 Mar · confirmed by reviewer	VERIFIED
Ownership	The leaver is effectively the only maintainer of the deploy workflow.	`deploy.yml` history: 31 of 33 commits by leaver	SUPPORTED

Risk	Image upload limit is unclear: meeting notes say 5 MB, code enforces 2 MB.	Notes, 2 Apr · `upload.ts` line 41	CONFLICTING
Next action	Finish PR #142 (RSVP export); blocked on review by the faculty coordinator.	PR #142 · review request	SUPPORTED
Contact	Owner of the Firebase project and its billing account.	`FIREBASE_*` keys in `.env.example` ; no owner found in any source	MISSING
Open question	The leaderboard feature may have been abandoned (no activity for 41 days).	Model inference only. Hidden from successor until reviewed.	INFERRED

WORKFLOW

06 Complete product workflow

Baton follows the brief's required five steps exactly. Each step has a clear owner and a clear output.

01 · CONNECT

Bring in sources

Admin links a GitHub repo and uploads notes, docs and a chat export. Each source is tagged with who may read it. Secrets are detected and stripped on ingestion.

Admin · leaver

02 · RECONSTRUCT

Build the ledger

Sources are split into timestamped chunks. The model extracts typed claims with quoted evidence. The repo checklist adds expected items. A timeline orders events.

Automatic

03 · VERIFY

Grade every claim

Each quote is matched against its source, then an entailment check tests whether the quote supports the claim. Rules assign one of the five statuses. Conflicts and gaps become questions.

Automatic · leaver answers gaps

04 · REVIEW

Human approves

Reviewer goes through the ledger with evidence side by side, then edits, rejects, confirms or promotes claims. A visibility preview shows the successor's view. Approval creates a version.

Reviewer

05 · HANDOVER

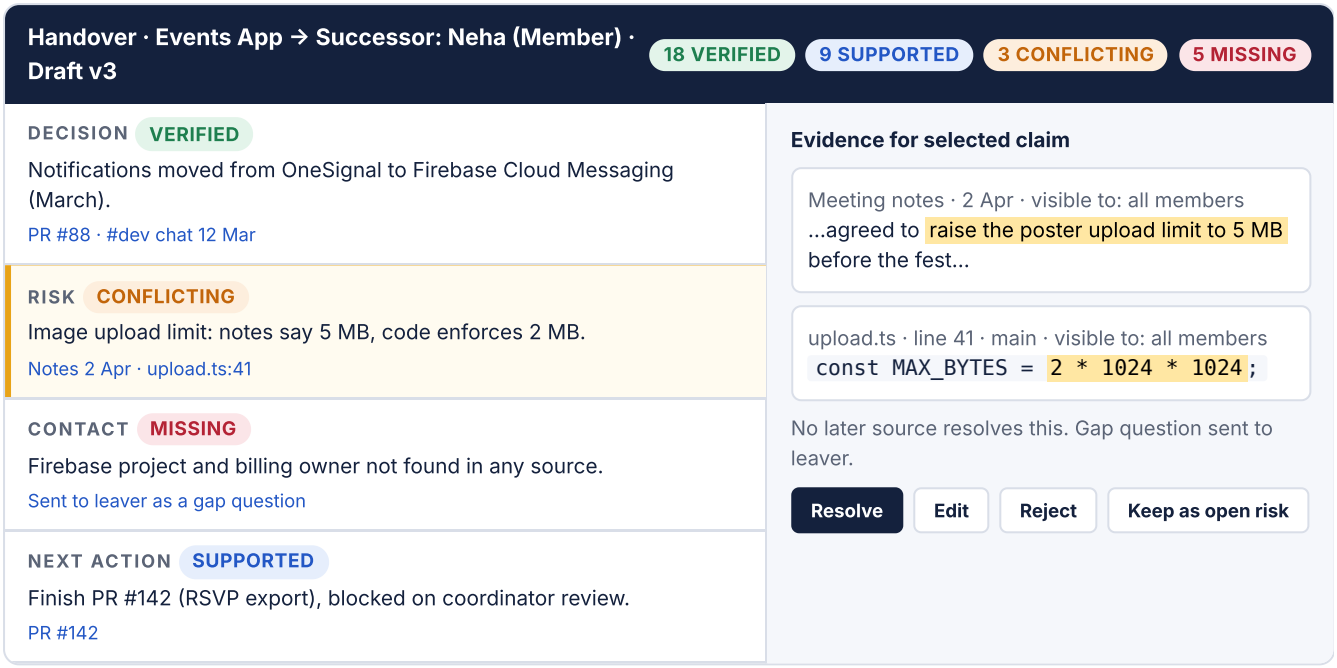
Continue the work

Successor gets a role-filtered plan: first actions, owners, risks, contacts and open questions, each with sources. They acknowledge items as they go.

Successor

Concept screen: the review step

A wireframe of the planned interface to show the interaction. No product code has been written yet.



BUILD

07 Main features

Scoped for one builder. The **Round 2 core** is the minimum needed for the full workflow to run end to end. Round 3 adds the three committed improvements. The cut list protects the core if time runs short.

Feature	What it does	Round
Source connectors	GitHub (public, or private via token): README and docs, commits, PRs, issues, review comments, CODEOWNERS, CI files. Upload Markdown, text and PDF notes. Import a Slack or Discord chat export (JSON).	R2 CORE
Access tagging	Every source and chunk carries an access list (members, roles). Access is checked before retrieval, not after. Default is deny.	R2 CORE
Secret scrubbing	API keys, tokens and passwords are detected on ingestion and replaced with a placeholder before anything is stored or sent to a model.	R2 CORE
Claim extraction	Typed claims with exact quoted spans, source ids and timestamps, returned as structured JSON.	R2 CORE
Expected-knowledge checklist	Derived from the repo (open PRs and issues by the leaver, env keys, workflows, files mostly maintained by the leaver). Uncovered items become Missing claims.	R2 CORE
Verification and status rules	Quote matching, entailment check, five-status assignment.	R2 CORE
Review workspace	Ledger with evidence panel. Edit, reject, confirm, promote. Version history and approval.	R2 CORE
Role-filtered handover	Successor view with first actions, owners, risks, contacts and open questions. Export to Markdown.	R2 CORE

Gap interview	Targeted questions from Missing and Conflicting claims, sent to the leaver (or the reviewer if the leaver is unavailable). Answers become cited testimony.	R3 · #1
Conflict and supersession timeline	Detects when a later decision reverses an earlier one versus a genuine contradiction. Shows a decision timeline.	R3 · #2
Acknowledgement and visibility preview	Successor acknowledges each item; open questions tracked to closure. Reviewer sees exactly what will be withheld and why, before approving.	R3 · #3

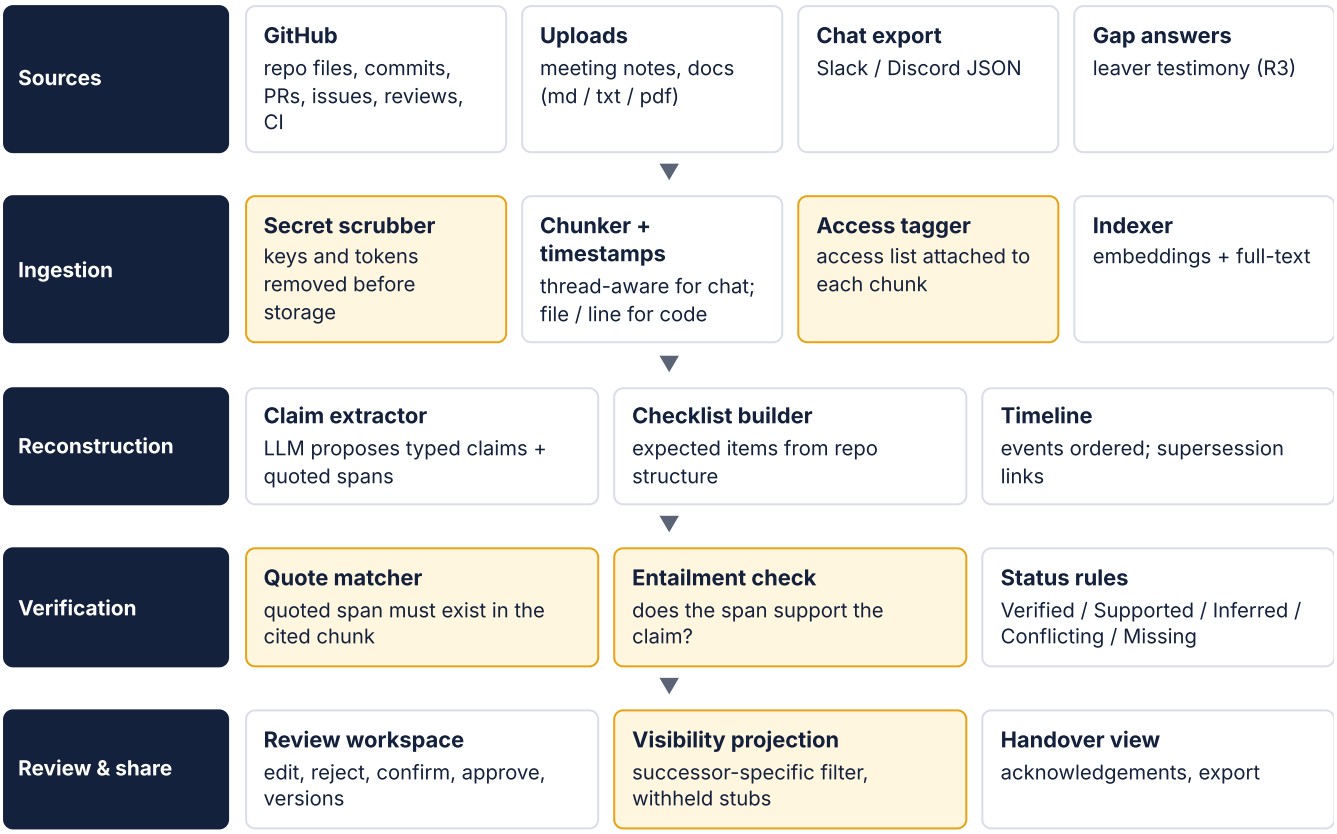
Cut list, in order, if time runs short: PDF upload (keep Markdown and text) → Discord export (keep Slack) → export formats beyond Markdown. The five-step core is never cut.

08 Technology stack and why

Layer	Choice	Why this choice
Frontend	Next.js (React, TypeScript), Tailwind CSS	Fast to build a review-heavy interface solo. Server components keep access checks on the server. Easy to deploy for judges.
Backend / AI pipeline	Python, FastAPI	The ingestion, NLP and verification libraries are all Python. One language for the whole AI pipeline means fewer moving parts.
Database	PostgreSQL with pgvector and full-text search	One database stores claims, evidence, access lists, versions and embeddings. Relational joins enforce access during retrieval. Hybrid keyword and vector search handles code identifiers better than vectors alone.
Language model	Hosted LLM API with JSON-schema structured output, behind a provider-agnostic adapter	Structured output makes claims machine-checkable. The adapter avoids lock-in and allows a local model later for confidential teams.
Verification	Fuzzy quote matching plus an open-source natural-language-inference cross-encoder running locally	A separate, cheap, deterministic-style check. The generating model never grades itself.
Embeddings	Small open-source sentence-embedding model, run locally	No per-call cost, and source text does not leave the server for indexing.
Auth and permissions	GitHub OAuth sign-in; role and access tables in Postgres	Matches where users already are. Access lists in the database are enough at this scale; OpenFGA (Zanzibar-style) is the documented path for larger teams [8].
Sources	GitHub REST API; Slack and Discord export JSON; file upload	Real data without app-store approvals. Exports avoid OAuth scope risk during the event.
Deployment	Docker Compose; frontend on Vercel, API and Postgres on a managed host	Gives judges a working link plus a reproducible local setup.

ARCHITECTURE

09 System architecture and data model



Highlighted boxes are the trust-critical components, where the brief's privacy and evidence requirements are enforced.

Core data model

Entity	Key fields
`Source`	type, uri, author, created_at, **acl** (members / roles)
`Chunk`	source_id, text, location (file + line, thread + message), timestamp, embedding, inherited acl
`Claim`	type, text, subject_key, **status**, created_at, superseded_by, origin (extracted / checklist / testimony)
`Evidence`	claim_id, chunk_id, quoted_span, match_score, entailment verdict
`ReviewEvent`	claim_id, reviewer, action (edit / reject / confirm / promote), before and after, timestamp
`Handover`	leaver, successor, scope, version, approved_by, approved_at
`Acknowledgement`	handover_id, claim_id, successor, state, timestamp

10 Major technical challenges and how we handle them

Challenge 1 — Making citations trustworthy (the hardest part)

Language models produce fluent citations that often do not support the claim [6]. **Approach:** (a) the extractor must return an *exact quoted span* plus the chunk id for every claim; (b) code checks that the span actually exists in that chunk, and rejects fabricated quotes; (c) a separate entailment model checks that the span supports the claim; (d) claims that fail are downgraded to `INFERRED`, not dropped silently. **How we will measure it:** a hand-labelled synthetic test set where we compute citation precision and recall in Round 2 and report the numbers.

Challenge 2 — Not leaking what the successor may not see

Filtering after retrieval is too late: a restricted chunk that reaches the prompt cannot be un-read [8]. **Approach:** access is enforced in the SQL retrieval query itself. At share time, each claim is projected against the successor's access. A claim is shown only if *all* its cited sources are visible to the successor; otherwise it is withheld (an optional stub, controlled by the reviewer). Round 3 adds re-rendering a claim from accessible sources only.

Challenge 3 — Telling a changed decision from a contradiction

Projects change their minds. A March decision reversed in June is not a conflict. **Approach:** claims about the same subject are grouped by a subject key. A later claim with explicit reversal language or a merged change supersedes the earlier one. Disagreements without that signal are flagged `CONFLICTING` and turned into a gap question, never resolved silently by the model.

TRUST

11 Privacy, security, traceability and human review

Permissions

- Default deny. Every chunk inherits its source's access list.
- Access is enforced in retrieval, again at share time, and previewed to the reviewer.
- Only authorised sources can be connected; the admin confirms authority when connecting.

Confidential data

- Secrets are scrubbed before storage or any model call.
- Personal phone numbers and emails are masked in shared views by default.
- Embeddings and verification run locally; only selected chunks go to the hosted LLM. A local-model mode is on the roadmap.

Traceability

- Every shared claim links to an exact location: file and line, PR comment, message, or note paragraph.
- Claims without evidence are labelled as such, never presented as fact.
- Every review action is logged and every approval is versioned.

Human control

- Nothing reaches the successor without reviewer approval.
- The reviewer can edit, reject, confirm and promote any claim, and must add a note to promote an Inferred claim.
- The leaver can see and challenge claims about their own work before approval.

Demo data. All demonstrations use a public open-source repository plus **clearly labelled synthetic** chat logs and meeting notes written for this project. No real workplace or club data will be uploaded.

12 Development plan across all three rounds

ROUND 1 · NOW
The Plan

- Problem research, user and scenario definition
- Workflow, claim-status model and architecture
- Scope and cut list fixed for a solo builder
- This proposal

ROUND 2 · BUILD WINDOW
The Core Loop

- **M1** Schema, auth, roles, workspace
- **M2** GitHub + upload + Slack export ingestion; secret scrubbing; access tagging
- **M3** Claim extraction, checklist, timeline
- **M4** Quote match + entailment + status rules
- **M5** Review workspace, versions, approval
- **M6** Successor view, deploy, test set, 3–5 user tests

ROUND 3 · FINISH
Three Committed Improvements

- **#1** Gap interview with cited testimony
- **#2** Conflict and supersession timeline
- **#3** Acknowledgements and visibility preview with re-rendering
- Reliability: failure cases, empty or sparse sources, permission tests
- Accessibility pass, docs, demo video

Round 2 definition of done: a judge can connect the demo repo and files, generate a ledger, open the evidence for any claim, edit and approve the handover, and view it as the successor, with nothing broken in between. All build work happens inside the official build window, with commit history as evidence.

Main risks

Risk	Mitigation
Solo bandwidth	Strict core-first plan, fixed cut list, single language for the AI pipeline, a single database.
LLM extraction quality varies	Status rules contain the damage: weak claims become Inferred and cannot be shared as fact. The labelled test set tracks improvement.
GitHub API rate limits, model cost	Authenticated requests, response caching, incremental ingestion, small models wherever possible.
Messy chat data	Thread-aware chunking; message ids kept for citation even when threads are reconstructed heuristically.

IMPACT

13 Expected impact

For successors
Find out where to start, who owns what and why decisions were made in one place, with sources to check, instead of weeks of asking around.

For leavers
A handover that asks only the questions the system genuinely cannot answer, rather than a blank document to fill during their last, busiest week.

For teams
Fewer abandoned projects after a key person leaves, fewer repeated debates over settled decisions, and a reviewable record of what was handed over.

How we will measure it (targets for Round 3 testing, not claims)

- **Citation precision** of shared claims on the labelled test set. Target: at least 90% of cited evidence actually supports its claim.

- **Leaver effort:** time the leaver spends answering gap questions. Target: under 30 minutes for the demo project.
- **Successor readiness:** testers answer 10 continuation questions about the demo project with and without Baton; we compare time taken and accuracy.
- **Permission safety:** zero restricted claims visible in successor views across permission test cases.

14 Future scope and known limitations

Future scope

- **Knowledge-risk radar:** warn teams *before* anyone leaves, using git ownership concentration ("bus factor") and the share of undocumented decisions per area.
- Live connectors with permission sync: Slack, Jira, Google Drive, Notion.
- Auto-drafted decision records (ADRs) from verified Decision claims.
- Local-model deployment for confidential organisations.
- Indic-language support for chat-heavy Indian teams.

Known limitations (stated honestly)

- Baton cannot recover knowledge that was never written down anywhere. The gap interview reduces this but cannot remove it.
- In the MVP, access lists are set in Baton rather than synced live from each tool.
- Output quality depends on how much the team wrote in its sources.
- English-only in the MVP.
- Automated statuses assist the reviewer; they do not replace them.

15 Team, role and contribution

Participant	Role	What I will personally build
Archit Dandavate Team LoneWolf · solo participant	Full-stack & AI engineer; product owner	Everything in this proposal: the data model and Postgres schema; GitHub, upload and chat-export ingestion with secret scrubbing and access tagging; the claim extraction and expected-knowledge checklist; the verification pipeline and status rules; the Next.js review workspace and successor view; deployment, testing, documentation and the demo.

Responsibilities. As a solo team, all responsibilities sit with me. I plan to split my build time roughly as: ingestion and access control 25%, AI pipeline and verification 35%, review and handover interface 25%, testing, deployment and documentation 15%.

DISCLOSURES

16 Disclosures and references

- **No code exists yet.** No product code, templates or boilerplate were written before the official build window. All development will be visible in the repository's commit history.
- **AI assistance:** an AI assistant (Claude, by Anthropic) was used to help with desk research and drafting of this proposal. The product itself will call a hosted LLM API, which will be named in the Round 2 document.
- **Planned open-source components** (used under their licences): Next.js, React, Tailwind CSS, FastAPI, PostgreSQL, pgvector, an open-source sentence-embedding model and an open-source NLI cross-encoder. Exact versions will be listed in Round 2.
- **Data:** public repositories and labelled synthetic data only.
- **Prior products considered:** Sensay and SkillPass (AI interviews of the leaver), Exit Insights (artifact-based transfer packages), EmpCloud (handover task tracking), Glean (permission-aware enterprise search) [11]. Baton differs by grading each claim's evidence, detecting missing knowledge from repository structure, and projecting visibility per successor. Microsoft retired Viva Topics, its AI knowledge-extraction product, in February 2025; AI-generated content no user had touched was removed at retirement [9], which supports keeping human review at the centre.

References

1. Panopto with YouGov, *Workplace Knowledge and Productivity Report* (survey of 1,001 US workers), 2018. Reported by HR Dive: hrdive.com/news/inefficient-knowledge-sharing-costs-large-us-businesses-47m-a-year/527892
2. Stack Overflow, *2024 Developer Survey — Professional Developers*. survey.stackoverflow.co/2024/professional-developers
3. O. Nourry, M. Kondo, S. Saito, Y. Iimura, N. Ubayashi, Y. Kamei, "Myth: The loss of core developers is a critical issue for OSS communities," arXiv:2412.00313, 2024; and IEICE Transactions on Information and Systems, 2025.
4. Company-reported voluntary attrition for TCS, Infosys, Wipro and HCLTech, FY26 quarterly results, as reported by People Matters and INDmoney.
5. R. Alkadhi, T. Lata, E. Guzman, B. Bruegge, "How do developers discuss rationale?", IEEE SANER, 2018.
6. N. F. Liu, T. Zhang, P. Liang, "Evaluating Verifiability in Generative Search Engines," Findings of EMNLP 2023. arXiv:2304.09848
7. "DRMiner: mining design rationale from issue logs with LLMs," arXiv:2405.19623, 2024.
8. OpenFGA documentation, "RAG Authorization." openfga.dev/docs/modeling/agents/rag-authorization
9. Microsoft Learn, "Frequently asked questions about changes coming to Topics." learn.microsoft.com/en-us/microsoft-365/topics/topics-changes-faq
10. IISE, "Succession Planning" (student chapter guidance). iise.org/Details.aspx?id=1124
11. Product websites: sensay.io; skillpasspro.com; exit-insights.com; empcloud.com; glean.com. Accessed November 2026.